

Lexical Analysis Programming Languages

Sujit Kumar Chakrabarti

IIITB

Language Processing

Compilers



Language Processing

Compilers

```
int    main    (    )    {    printf    (    Hello World!    )    ;    }
```

Compiler Phases

Lexical Analysis

Compiler Phases

Lexical Analysis



Implementation of Lexical Analysis

Example

Pattern	Specification
<i>ADDOP</i>	+
<i>MULOP</i>	*
<i>RPAREN</i>	)
<i>LPAREN</i>	(
<i>WS</i>	(' ' \t)*
<i>NUMBER</i>	(<i>digit</i>)+

Implementation of Lexical Analysis

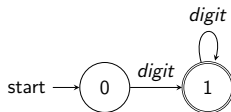
Transition Diagram

Example
NUMBER:

Implementation of Lexical Analysis

Transition Diagram

Example
NUMBER:



Regular Expressions

Example – Identifier

► English:

1. Must start with an alphabetic character
2. Subsequently, may have either an alphabetic character (α), a numeric character (β), separated by zero or more underscores ('_').

Regular Expressions

Example – Identifier

► English:

1. Must start with an alphabetic character
2. Subsequently, may have either an alphabetic character (α), a numeric character (β), separated by zero or more underscores ('_').

► Regular Expression: $\alpha\{[(\alpha|\beta)('_'\star)]\star(\alpha|\beta)\}?$

Regular Expressions

Rules of Construction

1. ϵ (empty symbol)
2. Let r and s be two regular expressions:
 - $r|s$ Choice
 - rs Concatenation
 - r^* Zero or more
 - r^+ One or more
 - $r?$ Zero or one

Regular Expressions

Activity

Regular expression for floating point numbers in C programming language

Regular Expressions

Regular Expressions and Finite State Automata

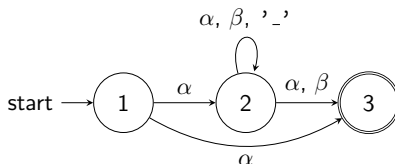
- ▶ Equivalent
- ▶ Let R be the set of all regular expressions, and let F be the set of all finite state automata.

$$\forall r \in R, \exists f \in F \text{ such that } L(r) = L(f)$$

Regular Expressions

Regular Expressions and Finite State Automata

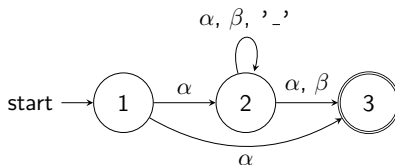
Example: For the regular expression $\alpha\{[(\alpha|\beta)('_'\star)]\star(\alpha|\beta)\}?$, the finite state automaton is:



Regular Expressions

Regular Expressions and Finite State Automata

Example: For the regular expression $\alpha\{[(\alpha|\beta)('_'\star)]\star(\alpha|\beta)\}?$, the finite state automaton is:



NFA or DFA?

Regular Expressions

Activity

FSA for floating point numbers in C programming language

Test Cases

For $\alpha((\alpha|\beta)(\cdot\cdot\star))\star(\alpha|\beta)$:

aab_b_baa	
aa1	
aa1_	
aa1_b	

Regular Expressions

Test Cases

For $\alpha((\alpha|\beta)('_*')) \star (\alpha|\beta)$:

aab_b_baa	✓
aa1	✓
aa1_	×
aa1_b	✓

Regular Expressions

Test Cases

For $\alpha((\alpha|\beta)('_*')) \star (\alpha|\beta)$:

aab_b_baa	✓
aa1	✓
aa1_	×
aa1_b	✓

- ▶ All could be good test cases.
 - ▶ valid inputs
 - ▶ invalid inputs

Implementation of Lexical Analysis

Transition Diagram

Example

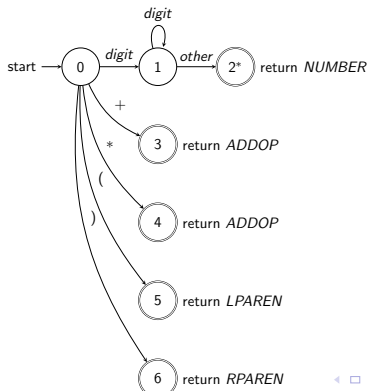
Complete Transition Diagram:

Implementation of Lexical Analysis

Transition Diagram

Example

Complete Transition Diagram:



Implementation of Lexical Analysis

Activity

Extend the discussed transition diagram to recognise the following token classes:

1. *LE* ($\leq$)
2. *NE* ($\neq$)
3. *LT* ($<$)
4. *ASSIGN* ($=$)
5. *EQ* ($==$)

Implementation of Lexical Analysis

Implementing the transition system

Implementation of Lexical Analysis

Implementating the transition system

1. How to deal with multi-character patterns?

Implementation of Lexical Analysis

Implementing the transition system

1. How to deal with multi-character patterns?
2. How to deal with the situation when a token is a prefix of another?

Implementation of Lexical Analysis

Implementing the transition system

1. How to deal with multi-character patterns?
2. How to deal with the situation when a token is a prefix of another?
3. How to deal with reserved keywords?

Implementation of Lexical Analysis

Manual implementation

1. Each token type is process by a separate FSA/scanner.
2. The lexer is an iterative algorithm. Each iteration processes one symbol from the input.

```
while (...) {  
    provide the next input symbol to each candidate scanner.  
    Compute result as per their output.  
}
```

3. Lexer output is based on the response of the scanners at the end of every iteration.

Implementation of Lexical Analysis

Manual implementation

FSA Response

1. Termination with failure: `State.Terminate(false)`
2. Termination with success: `State.Terminate(true)`
3. State transition: `State.State(_)`

Implementation of Lexical Analysis

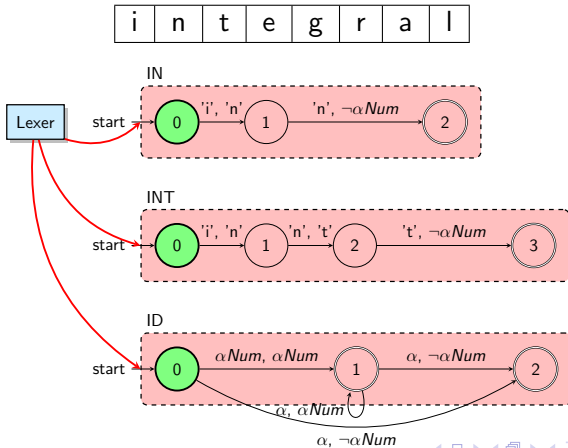
Manual implementation

Scanner Response Based on the Scanner response, the lexer does one of the following:

1. `State.Terminate(false)`: Remove from candidate list.
2. `State.Terminate(true)`: Take action as per the following:
 - ▶ If there are no other candidate scanners going into the next iteration, then return with the result.
 - ▶ Else (if there are other candidate scanners), the successful scanner is removed from the list and its output is ignored.
3. `State.State(_)`: Retain in the candidate list.

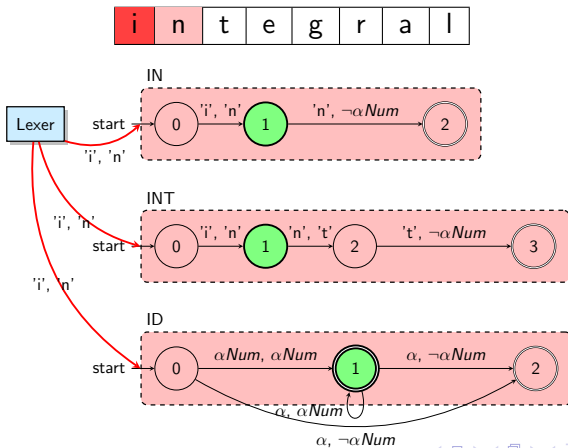
Implementation of Lexical Analysis

Manual Implementation of Lexical Analysers – Example



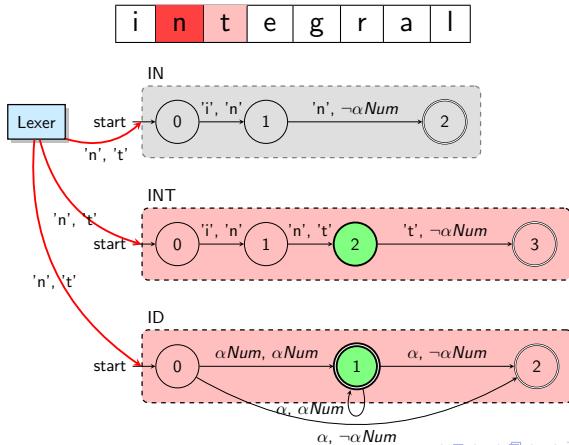
Implementation of Lexical Analysis

Manual Implementation of Lexical Analysers – Example



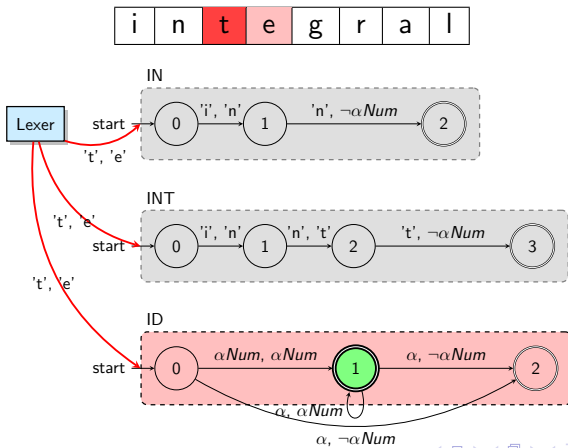
Implementation of Lexical Analysis

Manual Implementation of Lexical Analysers – Example



Implementation of Lexical Analysis

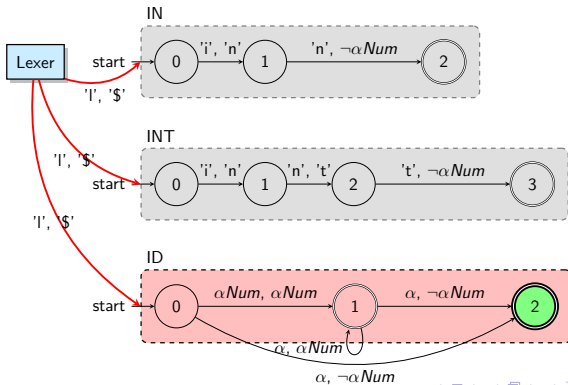
Manual Implementation of Lexical Analysers – Example



Implementation of Lexical Analysis

Manual Implementation of Lexical Analysers – Example

i	n	t	e	g	r	a	l
---	---	---	---	---	---	---	----------



Implementation of Lexical Analysis

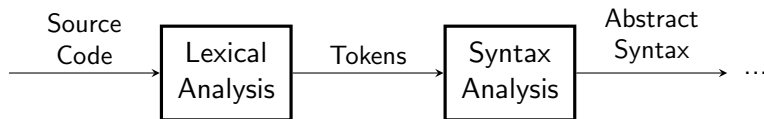
Manual Implementation of Lexical Analysers

Important Points

1. The scanners must be ordered carefully.
2. Alternative design.
 - ▶ No separate scanner for keywords
 - ▶ Maintain a table of keywords
 - ▶ On detecting an identifier, check if it's a keyword. If yes, return suitable token. Otherwise, return identifier.

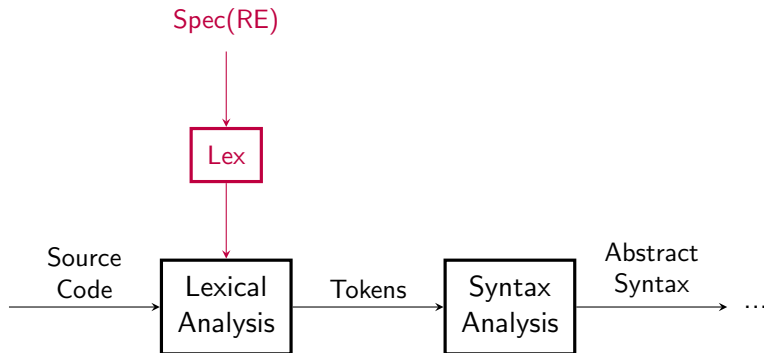
Implementation of Lexical Analysis

Automatic Generation of Lexical Analysers



Implementation of Lexical Analysis

Automatic Generation of Lexical Analysers



Implementation of Lexical Analysis

Automatic Generation of Lexical Analysers

